

Register Number:192425325

Name:B.Archana

CO3-AT-1

MLA0201-Fundamentals Of Machine Learning

Experiment -1:

Customer Segmentation using K-Means and PCA: Load the Mall Customers dataset and preprocess the data. Apply the K-Means algorithm to identify customer clusters, then use PCA to reduce the data to two dimensions and visualize the clusters. Analyze the optimal number of clusters using the Elbow Method and Silhouette Score.

Customer Segmentation using K-Means and PCA

Aim:

To perform customer segmentation using the K-Means clustering algorithm on the Mall Customers dataset, determine the optimal number of clusters using the Elbow Method and Silhouette Score, and use PCA to reduce the data to two dimensions for cluster visualization.

Dataset:

Dataset: Mall Customer Segmentation Data

Source: Kaggle

The dataset contains customer information such as Customer ID, Gender, Age, Annual Income, and Spending Score.

Kaggle Dataset Link:

<https://www.kaggle.com/datasets/vjchoudhary7/customer-segmentation-tutorial-in-python>

Procedure:

1. Load the Mall Customers dataset.
2. Read the dataset using Pandas.
3. Select relevant numerical features: Age, Annual Income, and Spending Score.
4. Standardize the selected features using StandardScaler.
5. Apply K-Means clustering for different values of K.
6. Calculate the Within-Cluster Sum of Squares (WCSS) for the Elbow Method.
7. Calculate the Silhouette Score for each K.
8. Select the optimal number of clusters.

9. Apply K-Means using the selected K value.
10. Apply **PCA** to reduce the features to two dimensions.
11. Visualize the customer clusters using a scatter plot.
12. Display the cluster distribution and final result.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA
# Load dataset
# Upload Mall_Customers.csv to Google Colab first
from google.colab import files
uploaded = files.upload()
file_name = list(uploaded.keys())[0]
df = pd.read_csv(file_name)
print("Dataset Shape:", df.shape)
print("\nFirst 5 Rows:")
print(df.head())
# Select numerical features
features = ['Age', 'Annual Income (k$)', 'Spending Score (1-100)']
X = df[features]
# Standardize data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# Elbow Method and Silhouette Score
wcss = []
silhouette_scores = []
k_values = range(2, 11)
for k in k_values:
```

```

model = KMeans(n_clusters=k, random_state=42, n_init=10)
labels = model.fit_predict(X_scaled)
wcss.append(model.inertia_)
silhouette_scores.append(silhouette_score(X_scaled, labels))

# Display scores
results = pd.DataFrame({
    'K': list(k_values),
    'WCSS': wcss,
    'Silhouette Score': silhouette_scores
})

print("\nElbow and Silhouette Results:")
print(results.round(4).to_string(index=False))

# Find best K using Silhouette Score
best_k = k_values[np.argmax(silhouette_scores)]
print("\nOptimal Number of Clusters:", best_k)
print("Highest Silhouette Score:", round(max(silhouette_scores), 4))

# Elbow plot
plt.figure(figsize=(7, 4))
plt.plot(k_values, wcss, marker='o')
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.grid(True)
plt.show()

# Silhouette plot
plt.figure(figsize=(7, 4))
plt.plot(k_values, silhouette_scores, marker='o')
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Silhouette Score")

```

```

plt.title("Silhouette Score")
plt.grid(True)
plt.show()

# Apply K-Means with optimal K
kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=10)
df['Cluster'] = kmeans.fit_predict(X_scaled)

# Cluster distribution
print("\nCustomer Count in Each Cluster:")
print(df['Cluster'].value_counts().sort_index())

# PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# PCA visualization
plt.figure(figsize=(8, 5))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=df['Cluster'], cmap='viridis', s=50)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Customer Segmentation using K-Means and PCA")
plt.colorbar(label="Cluster")
plt.grid(True)
plt.show()

# Final cluster centers
print("\nCluster Centers:")
print(pd.DataFrame(
    scaler.inverse_transform(kmeans.cluster_centers_),
    columns=features
).round(2))

```

Output:

... Mall_Customers.csv
Mall_Customers.csv(text/csv) - 3981 bytes, last modified: 8/12/2026 - 100% done
Saving Mall_Customers.csv to Mall_Customers.csv
Dataset Shape: (200, 5)

First 5 Rows:

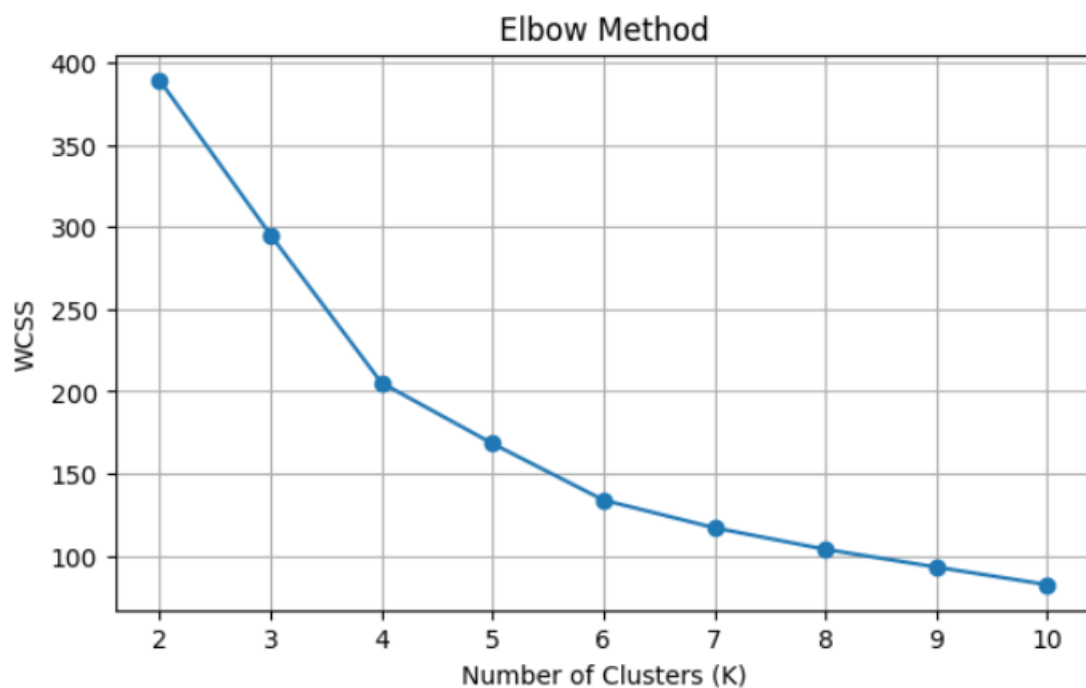
	CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

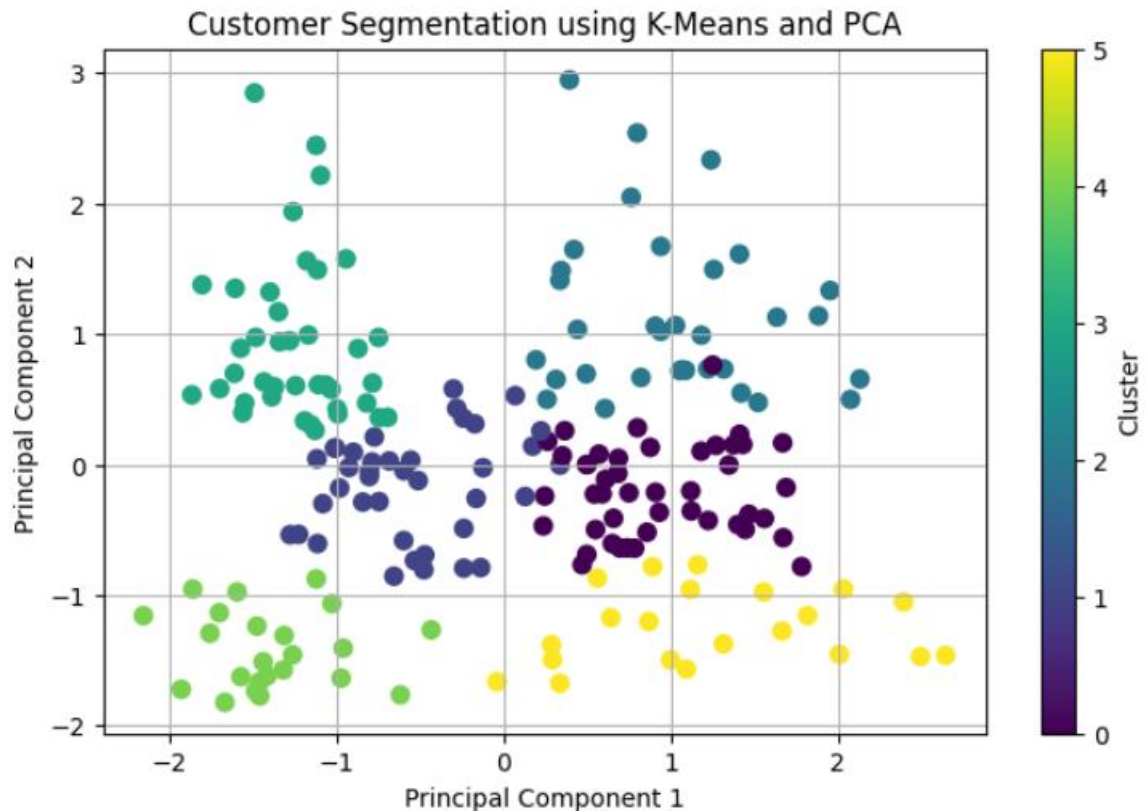
Elbow and Silhouette Results:

K	WCSS	Silhouette Score
2	389.3862	0.3355
3	295.2122	0.3578
4	205.2251	0.4040
5	168.2476	0.4166
6	133.8684	0.4284
7	117.0116	0.4172
8	103.8733	0.4082
9	93.0929	0.4177
10	82.3852	0.4066

Optimal Number of Clusters: 6
Highest Silhouette Score: 0.4284

Optimal Number of Clusters: 6
Highest Silhouette Score: 0.4284





Result:

Output obtained from the program:

- The dataset is successfully loaded and preprocessed.
- The **Elbow Method** is used to identify the point where WCSS starts decreasing slowly.
- The **Silhouette Score** is calculated for $K = 2$ to 10.
- The program automatically selects the K value having the **highest Silhouette Score**.
- K-Means assigns every customer to a cluster.
- PCA reduces the three-dimensional customer data into **two principal components**.
- The final scatter plot clearly shows the different customer segments.

Experiment -2:

Clustering using EM Algorithm and Gaussian Mixture Model: Load the Digits dataset from Scikit-learn and preprocess the features. Implement the Gaussian Mixture Model (EM Algorithm) to cluster the data and compare its performance with K-Means using appropriate clustering evaluation metrics. Visualize the clusters after applying PCA.

Aim:

To cluster the Digits dataset using the Gaussian Mixture Model (GMM) based on the Expectation-Maximization (EM) algorithm, compare its performance with K-Means using clustering evaluation metrics, and visualize the clusters using PCA.

Dataset :

MNIST Handwritten Digits

<https://www.kaggle.com/datasets/oddrational/mnist-in-csv>

Procedure:

1. Load the Digits dataset using Scikit-learn.
2. Extract the 64 pixel features.
3. Standardize the features using StandardScaler.
4. Apply the Gaussian Mixture Model (GMM) using the EM algorithm.
5. Apply K-Means clustering on the same dataset.
6. Calculate the Silhouette Score and Adjusted Rand Index (ARI) for both models.
7. Compare the performance of GMM and K-Means.
8. Apply PCA to reduce the 64-dimensional data to 2 dimensions.
9. Plot the clusters obtained from GMM.
10. Plot the clusters obtained from K-Means.
11. Compare the clustering results and identify the better-performing method.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
```

```

from sklearn.mixture import GaussianMixture
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
from sklearn.metrics import adjusted_rand_score
import warnings
warnings.filterwarnings("ignore")
print("UPLOAD mnist_train.csv")
uploaded=files.upload()
if len(uploaded)==0:
    raise Exception("Please upload mnist_train.csv")
file_name=list(uploaded.keys())[0]
print("File selected:",file_name)
df=pd.read_csv(file_name)
print("Dataset loaded successfully")
print("Original shape:",df.shape)
print(df.head())
if "label" not in df.columns:
    raise Exception("The uploaded file must contain a label column.")
y=df["label"]
X=df.drop(columns=["label"])
X=X.select_dtypes(include=[np.number])
X=X.replace([np.inf,-np.inf],np.nan)
X=X.fillna(X.median())
print("Number of samples:",X.shape[0])
print("Number of features:",X.shape[1])
print("Classes:",sorted(y.unique()))
print("Preparing data for clustering")
sample_size=min(10000,len(X))
rng=np.random.RandomState(42)
indices=rng.choice(len(X),sample_size,replace=False)
X=X.iloc[indices].reset_index(drop=True)
y=y.iloc[indices].reset_index(drop=True)

```



```
print("Samples used:",len(X))
scaler=StandardScaler()
X_scaled=scaler.fit_transform(X)
print("Standardization completed")
print("K-MEANS CLUSTERING")
kmeans=KMeans(n_clusters=10,random_state=42,n_init=10)
kmeans_labels=kmeans.fit_predict(X_scaled)
kmeans_silhouette=silhouette_score(X_scaled,kmeans_labels)
kmeans_ari=adjusted_rand_score(y,kmeans_labels)
print("K-Means Silhouette Score:",round(kmeans_silhouette,4))
print("K-Means Adjusted Rand Index:",round(kmeans_ari,4))
print("GAUSSIAN MIXTURE MODEL")
gmm=GaussianMixture(n_components=10,covariance_type="diag",random_state=42,n_init=
2,max_iter=100)
gmm_labels=gmm.fit_predict(X_scaled)
gmm_silhouette=silhouette_score(X_scaled,gmm_labels)
gmm_ari=adjusted_rand_score(y,gmm_labels)
print("GMM Silhouette Score:",round(gmm_silhouette,4))
print("GMM Adjusted Rand Index:",round(gmm_ari,4))
print("PCA VISUALIZATION")
pca=PCA(n_components=2,random_state=42)
X_pca=pca.fit_transform(X_scaled)
print("PCA shape:",X_pca.shape)
print("Variance explained:",round(pca.explained_variance_ratio_.sum()*100,2),"%")
plt.figure(figsize=(9,7))
plt.scatter(X_pca[:,0],X_pca[:,1],c=kmeans_labels,cmap="tab10",s=12,alpha=0.6)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("K-Means Clusters after PCA")
plt.colorbar(label="Cluster")
plt.grid(True)
plt.show()
```

```

plt.figure(figsize=(9,7))
plt.scatter(X_pca[:,0],X_pca[:,1],c=gmm_labels,cmap="tab10",s=12,alpha=0.6)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Gaussian Mixture Model Clusters after PCA")
plt.colorbar(label="Cluster")
plt.grid(True)
plt.show()

plt.figure(figsize=(9,7))
plt.scatter(X_pca[:,0],X_pca[:,1],c=y,cmap="tab10",s=12,alpha=0.6)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Actual Digit Labels after PCA")
plt.colorbar(label="Digit")
plt.grid(True)
plt.show()

comparison=pd.DataFrame({"Method":["K-Means","GaussianMixture
Model"],"Clusters":[10,10],"Silhouette
Score":[kmeans_silhouette,gmm_silhouette],"Adjusted Rand Index":[kmeans_ari,gmm_ari]})
print("MODEL COMPARISON")
print(comparison.round(4))
print("K-MEANS CLUSTER COUNTS")
print(pd.Series(kmeans_labels).value_counts().sort_index())
print("GMM CLUSTER COUNTS")
print(pd.Series(gmm_labels).value_counts().sort_index())
print("FINAL RESULT")
print("K-Means Silhouette Score:",round(kmeans_silhouette,4))
print("GMM Silhouette Score:",round(gmm_silhouette,4))
print("K-Means ARI:",round(kmeans_ari,4))
print("GMM ARI:",round(gmm_ari,4))
print("K-Means performs clustering using centroid distance.")
print("GMM performs probabilistic clustering using the EM algorithm.")
print("PCA reduces the high-dimensional pixel data to two dimensions.")

```

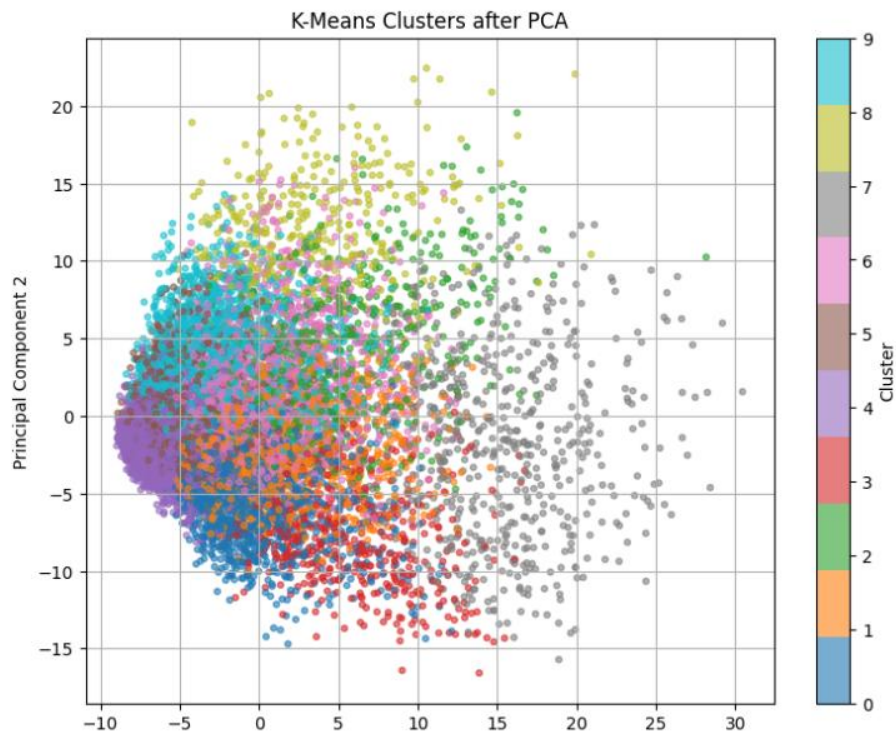
```
print("Silhouette Score measures cluster separation and compactness.")
print("Adjusted Rand Index compares predicted clusters with actual digit labels.")
print("Experiment completed successfully.")
```

Output:

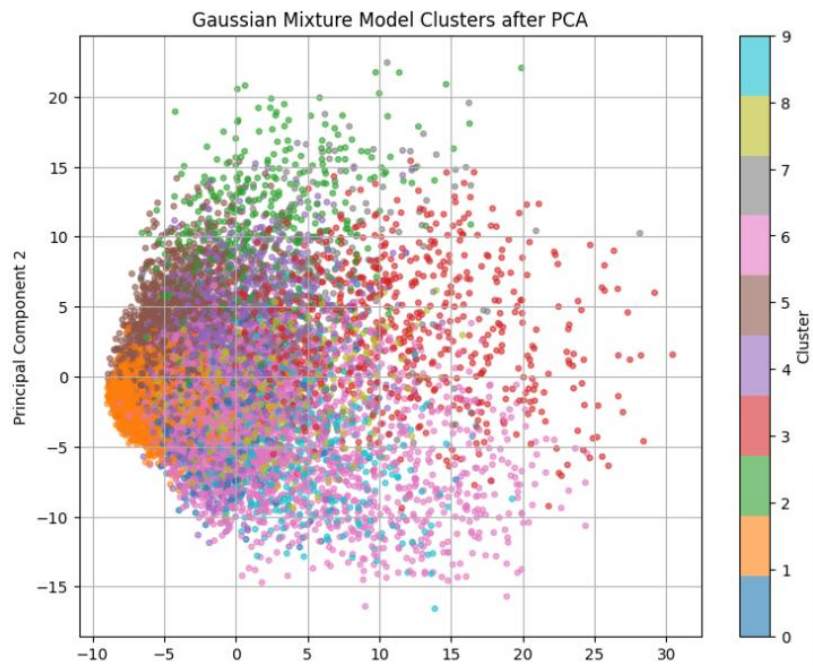
```
... UPLOAD mnist_train.csv
  Choose Files  mnist_train.csv
mnist_train.csv(text/csv) - 109640201 bytes, last modified: 8/13/2026 - 100% done
Saving mnist_train.csv to mnist_train.csv
File selected: mnist_train.csv
Dataset loaded successfully
Original shape: (60000, 785)
  label  1x1  1x2  1x3  1x4  1x5  1x6  1x7  1x8  1x9  ...  28x19  28x20  \
0      5    0    0    0    0    0    0    0    0    0  ...    0    0
1      0    0    0    0    0    0    0    0    0    0  ...    0    0
2      4    0    0    0    0    0    0    0    0    0  ...    0    0
3      1    0    0    0    0    0    0    0    0    0  ...    0    0
4      9    0    0    0    0    0    0    0    0    0  ...    0    0

      28x21  28x22  28x23  28x24  28x25  28x26  28x27  28x28
0      0      0      0      0      0      0      0      0
1      0      0      0      0      0      0      0      0
2      0      0      0      0      0      0      0      0
3      0      0      0      0      0      0      0      0
4      0      0      0      0      0      0      0      0

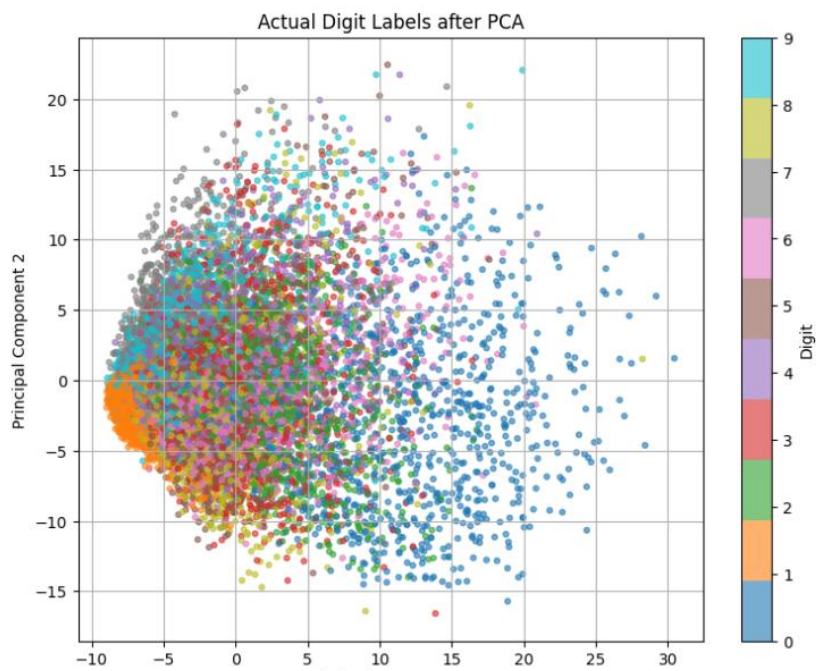
[5 rows x 785 columns]
Number of samples: 60000
Number of features: 784
Classes: [np.int64(0), np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(
Preparing data for clustering
Samples used: 10000
Standardization completed
K-MEANS CLUSTERING
```



...



...



MODEL COMPARISON

	Method	Clusters	Silhouette Score	Adjusted Rand Index
0	K-Means	10	0.0044	0.3211
1	Gaussian Mixture Model	10	-0.0416	0.1932

K-MEANS CLUSTER COUNTS

0 1256
1 947
2 612
3 489
4 1777
5 1021
6 1463
7 603
8 384
9 1448

Name: count, dtype: int64

GMM CLUSTER COUNTS

0 351
1 1732
2 408
3 812
4 1493
5 1335
6 2521
7 319
8 567
9 462

Name: count, dtype: int64

FINAL RESULT

K-Means Silhouette Score: 0.0044

```

-
---
Name: count, dtype: int64
FINAL RESULT
K-Means Silhouette Score: 0.0044
GMM Silhouette Score: -0.0416
K-Means ARI: 0.3211
GMM ARI: 0.1932
K-Means performs clustering using centroid distance.
GMM performs probabilistic clustering using the EM algorithm.
PCA reduces the high-dimensional pixel data to two dimensions.
Silhouette Score measures cluster separation and compactness.
Adjusted Rand Index compares predicted clusters with actual digit labels.
Experiment completed successfully.
```

Result:

The Gaussian Mixture Model using the EM algorithm and K-Means were successfully applied to the Digits dataset. Their performance was compared using Silhouette Score and Adjusted Rand Index, and PCA was used to visualize the resulting clusters in two dimensions.

Experiment -3:

Dimensionality Reduction using PCA, Factor Analysis, and ICA:

Load the Wine dataset from Scikit-learn and standardize the features. Apply Principal Component Analysis (PCA), Factor Analysis (FA), and Independent Component Analysis (ICA) to reduce the dimensionality and compare their outputs. Discuss how these techniques help address the Curse of Dimensionality through visualizations and component analysis.

Aim:

To reduce the dimensionality of the Wine dataset using Principal Component Analysis (PCA), Factor Analysis (FA), and Independent Component Analysis (ICA), compare their outputs, and visualize the reduced components to understand how dimensionality reduction helps overcome the Curse of Dimensionality.

Dataset:

Dataset: <https://www.kaggle.com/datasets/uciml/red-wine-quality-cortez-et-al-2009>

The Wine dataset contains **178 samples, 13 features, and 3 classes**. The features represent different chemical properties of wine.

Dataset name: Wine Quality – Red Wine

Procedure:

1. Load the Wine dataset from Scikit-learn.
2. Separate the features and target classes.
3. Standardize all features using StandardScaler.
4. Apply PCA and reduce the data to 2 components.
5. Apply Factor Analysis and reduce the data to 2 components.
6. Apply ICA and reduce the data to 2 components.

7. Display the explained variance ratio for PCA.
8. Visualize the outputs of PCA, FA, and ICA.
9. Compare the three dimensionality-reduction techniques.
10. Analyze how dimensionality reduction helps overcome the Curse of Dimensionality.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA, FactorAnalysis, FastICA
import warnings
warnings.filterwarnings("ignore")
print("UPLOAD winequality-red.csv")
uploaded=files.upload()
if len(uploaded)==0:
    raise Exception("No file was uploaded.")
file_name=list(uploaded.keys())[0]
print("File selected:",file_name)
df=pd.read_csv(file_name,sep=None,engine="python")
print("DATASET LOADED")
print("Rows:",df.shape[0])
print("Columns:",df.shape[1])
print(df.head())
print("Column names:",df.columns.tolist())
print("Missing values:",df.isnull().sum().sum())
df.info()
if "quality" in df.columns:
    target="quality"
else:
    target=df.columns[-1]
```

```

print("Target column:",target)
X=df.drop(columns=[target])
y=df[target]
X=X.select_dtypes(include=[np.number])
print("Number of samples:",X.shape[0])
print("Number of features:",X.shape[1])
print("Features:",X.columns.tolist())
X=X.replace([np.inf,-np.inf],np.nan)
X=X.fillna(X.median())
print("Missing values after cleaning:",X.isnull().sum().sum())
scaler=StandardScaler()
X_scaled=scaler.fit_transform(X)
print("Standardization completed.")
print("Standardized data shape:",X_scaled.shape)
print("PCA ANALYSIS")
pca_all=PCA()
X_pca_all=pca_all.fit_transform(X_scaled)
variance=pca_all.explained_variance_ratio_
cumulative=np.cumsum(variance)
print("Explained variance by component:")
for i in range(len(variance)):
    print("PC",i+1,":",round(variance[i]*100,2),"%")
print("Total variance:",round(cumulative[-1]*100,2),"%")
plt.figure(figsize=(8,5))
plt.plot(range(1,len(cumulative)+1),cumulative,marker="o")
plt.xlabel("Number of Components")
plt.ylabel("Cumulative Explained Variance")
plt.title("PCA Explained Variance")
plt.grid(True)
plt.show()
pca=PCA(n_components=2)
X_pca=pca.fit_transform(X_scaled)

```

```

print("PCA reduced shape:",X_pca.shape)

print("Variance retained by 2 PCA
components:",round(pca.explained_variance_ratio_.sum()*100,2),"%")

pca_loadings=pd.DataFrame(pca.components_.T,index=X.columns,columns=["PC1","PC2"])

print("PCA Component Loadings:")

print(pca_loadings.round(4))

quality_values=y.to_numpy()

plt.figure(figsize=(8,6))

plt.scatter(X_pca[:,0],X_pca[:,1],c=quality_values,cmap="viridis",s=35,alpha=0.7)

plt.xlabel("Principal Component 1")

plt.ylabel("Principal Component 2")

plt.title("PCA - Wine Quality Dataset")

plt.colorbar(label="Wine Quality")

plt.grid(True)

plt.show()

print("FACTOR ANALYSIS")

fa=FactorAnalysis(n_components=2,random_state=42)

X_fa=fa.fit_transform(X_scaled)

print("FA reduced shape:",X_fa.shape)

fa_loadings=pd.DataFrame(fa.components_.T,index=X.columns,columns=["Factor1","Factor
2"])

print("Factor Loadings:")

print(fa_loadings.round(4))

plt.figure(figsize=(8,6))

plt.scatter(X_fa[:,0],X_fa[:,1],c=quality_values,cmap="viridis",s=35,alpha=0.7)

plt.xlabel("Factor 1")

plt.ylabel("Factor 2")

plt.title("Factor Analysis - Wine Quality Dataset")

plt.colorbar(label="Wine Quality")

plt.grid(True)

plt.show()

print("INDEPENDENT COMPONENT ANALYSIS")

```



```

ica=FastICA(n_components=2,random_state=42,max_iter=2000,tol=0.001,whiten="unit-
variance")

X_ica=ica.fit_transform(X_scaled)

print("ICA reduced shape:",X_ica.shape)

ica_components=pd.DataFrame(ica.components_,columns=X.columns,index=["IC1","IC2"])

print("ICA Component Matrix:")

print(ica_components.round(4))

plt.figure(figsize=(8,6))

plt.scatter(X_ica[:,0],X_ica[:,1],c=quality_values,cmap="viridis",s=35,alpha=0.7)

plt.xlabel("Independent Component 1")

plt.ylabel("Independent Component 2")

plt.title("ICA - Wine Quality Dataset")

plt.colorbar(label="Wine Quality")

plt.grid(True)

plt.show()

print("COMPARISON")

comparison=pd.DataFrame({"Technique":["Original","PCA","Factor
Analysis","ICA"],"Dimensions":[X.shape[1],X_pca.shape[1],X_fa.shape[1],X_ica.shape[1]]}
)

print(comparison)

fig,axes=plt.subplots(1,3,figsize=(18,5))

axes[0].scatter(X_pca[:,0],X_pca[:,1],c=quality_values,cmap="viridis",s=25)

axes[0].set_title("PCA")

axes[0].set_xlabel("PC1")

axes[0].set_ylabel("PC2")

axes[1].scatter(X_fa[:,0],X_fa[:,1],c=quality_values,cmap="viridis",s=25)

axes[1].set_title("Factor Analysis")

axes[1].set_xlabel("Factor 1")

axes[1].set_ylabel("Factor 2")

axes[2].scatter(X_ica[:,0],X_ica[:,1],c=quality_values,cmap="viridis",s=25)

axes[2].set_title("ICA")

axes[2].set_xlabel("IC1")

axes[2].set_ylabel("IC2")

```

```

plt.suptitle("PCA vs Factor Analysis vs ICA")
plt.tight_layout()
plt.show()
print("FINAL RESULT")
print("Original dimensions:",X.shape[1])
print("PCA dimensions:",X_pca.shape[1])
print("FA dimensions:",X_fa.shape[1])
print("ICA dimensions:",X_ica.shape[1])
print("PCA variance retained:",round(pca.explained_variance_ratio_.sum()*100,2),"%")
print("PCA: Finds directions of maximum variance.")
print("Factor Analysis: Finds underlying latent factors.")
print("ICA: Finds statistically independent components.")
print("Curse of Dimensionality: The original features are reduced to only 2 dimensions.")
print("This reduces complexity and makes visualization easier.")
print("EXPERIMENT COMPLETED SUCCESSFULLY")

```

Output:

```

... =====
Choose Files winequality-red.csv
winequality-red.csv(text/csv) - 100951 bytes, last modified: 8/13/2026 - 100% done
Saving winequality-red.csv to winequality-red.csv
File selected: winequality-red.csv

=====
DATASET LOADED
=====
Rows: 1599
Columns: 12

First 5 rows:
    fixed acidity  volatile acidity  citric acid  residual sugar  chlorides \
0           7.4           0.70           0.00           1.9           0.076
1           7.8           0.88           0.00           2.6           0.098
2           7.8           0.76           0.04           2.3           0.092
3          11.2           0.28           0.56           1.9           0.075
4           7.4           0.70           0.00           1.9           0.076

    free sulfur dioxide  total sulfur dioxide  density  pH  sulphates \
0           11.0           34.0  0.9978  3.51  0.56
1           25.0           67.0  0.9968  3.20  0.68
2           15.0           54.0  0.9970  3.26  0.65
3           17.0           60.0  0.9980  3.16  0.58
4           11.0           34.0  0.9978  3.51  0.56

```

```

***      alcohol  quality
0         9.4         5
1         9.8         5
2         9.8         5
3         9.8         6
4         9.4         5

```

Column names:

```
['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar']
```

Missing values:

```
0
```

Dataset information:

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 1599 entries, 0 to 1598
```

Data columns (total 12 columns):

#	Column	Non-Null Count	Dtype
0	fixed acidity	1599 non-null	float64
1	volatile acidity	1599 non-null	float64
2	citric acid	1599 non-null	float64
3	residual sugar	1599 non-null	float64
4	chlorides	1599 non-null	float64
5	free sulfur dioxide	1599 non-null	float64
6	total sulfur dioxide	1599 non-null	float64
7	density	1599 non-null	float64
8	pH	1599 non-null	float64
9	sulphates	1599 non-null	float64
10	alcohol	1599 non-null	float64
11	quality	1599 non-null	int64

dtypes: float64(11), int64(1)

memory usage: 150.0 KB

Target column: quality

```
=====
```

FEATURE INFORMATION

```
=====
```

Number of samples: 1599

Number of features: 11

Features:

```
['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar', 'chlorides',
```

Missing values after cleaning:

```
0
```

```
=====
```

STANDARDIZATION

```
=====
```

Standardization completed.

Standardized data shape:

```
(1599, 11)
```

```
=====
```

PCA ANALYSIS

```
=====
```

Explained variance by component:

```
PC 1 : 28.17 %
```

```
PC 2 : 17.51 %
```

```
PC 3 : 14.1 %
```

```
PC 4 : 11.03 %
```

```
PC 5 : 8.72 %
```

```
PC 6 : 6.0 %
```

```
PC 7 : 5.31 %
```

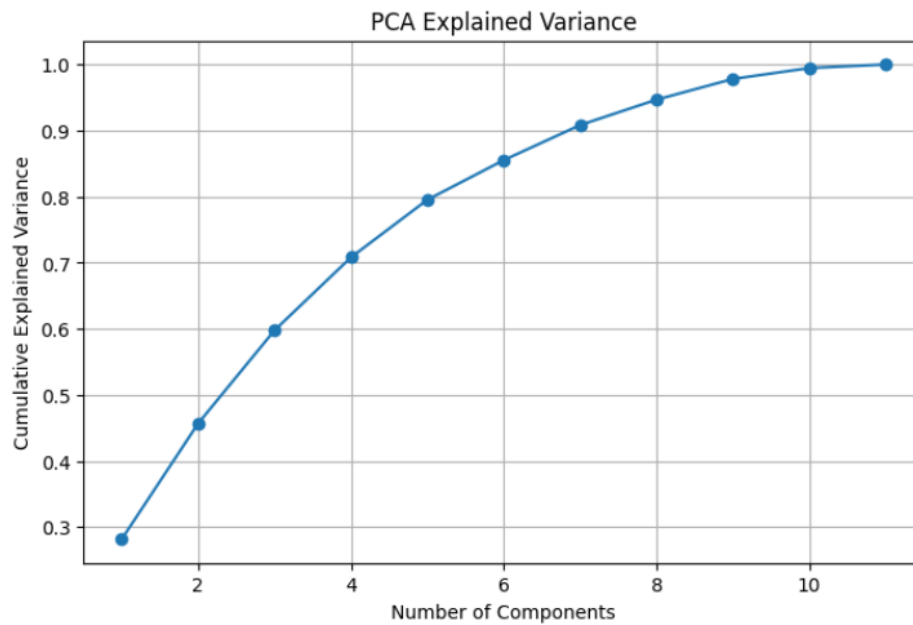
```
PC 8 : 3.85 %
```

```
PC 9 : 3.13 %
```

```
PC 10 : 1.65 %
```

```
PC 11 : 0.54 %
```

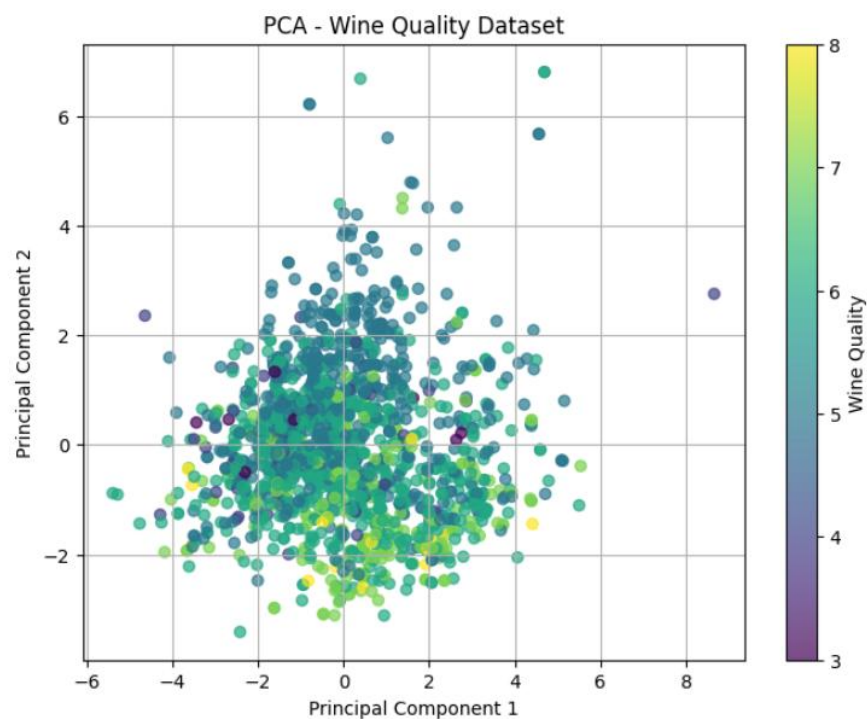
Total variance: 100.0 %

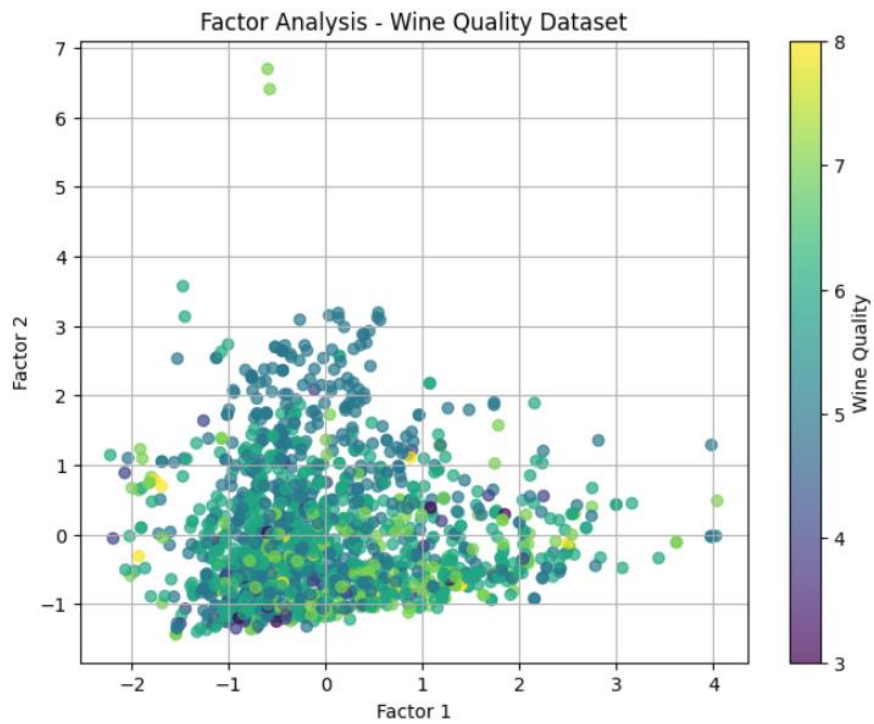


PCA reduced shape:
 (1599, 2)
 Variance retained by 2 PCA components: 45.68 %

PCA Component Loadings:

	PC1	PC2
fixed acidity	0.4893	-0.1105
volatile acidity	-0.2386	0.2749
citric acid	0.4636	-0.1518
residual sugar	0.1461	0.2721
chlorides	0.2122	0.1481
free sulfur dioxide	-0.0362	0.5136
total sulfur dioxide	0.0236	0.5695
density	0.3954	0.2336
pH	-0.4385	0.0067
sulphates	0.2429	-0.0376
alcohol	-0.1132	-0.3862





=====

INDEPENDENT COMPONENT ANALYSIS

=====

ICA reduced shape:

(1599, 2)

ICA Component Matrix:

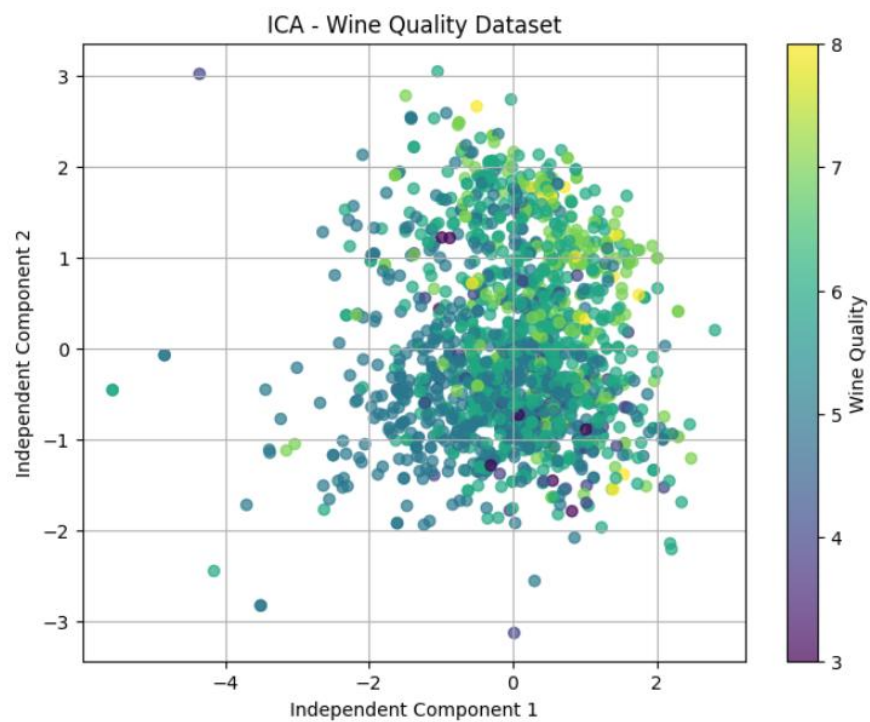
	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	\
IC1	-0.0856	-0.0915	-0.0527	-0.2095	-0.1553	
IC2	0.2762	-0.2219	0.2803	-0.0379	0.0424	

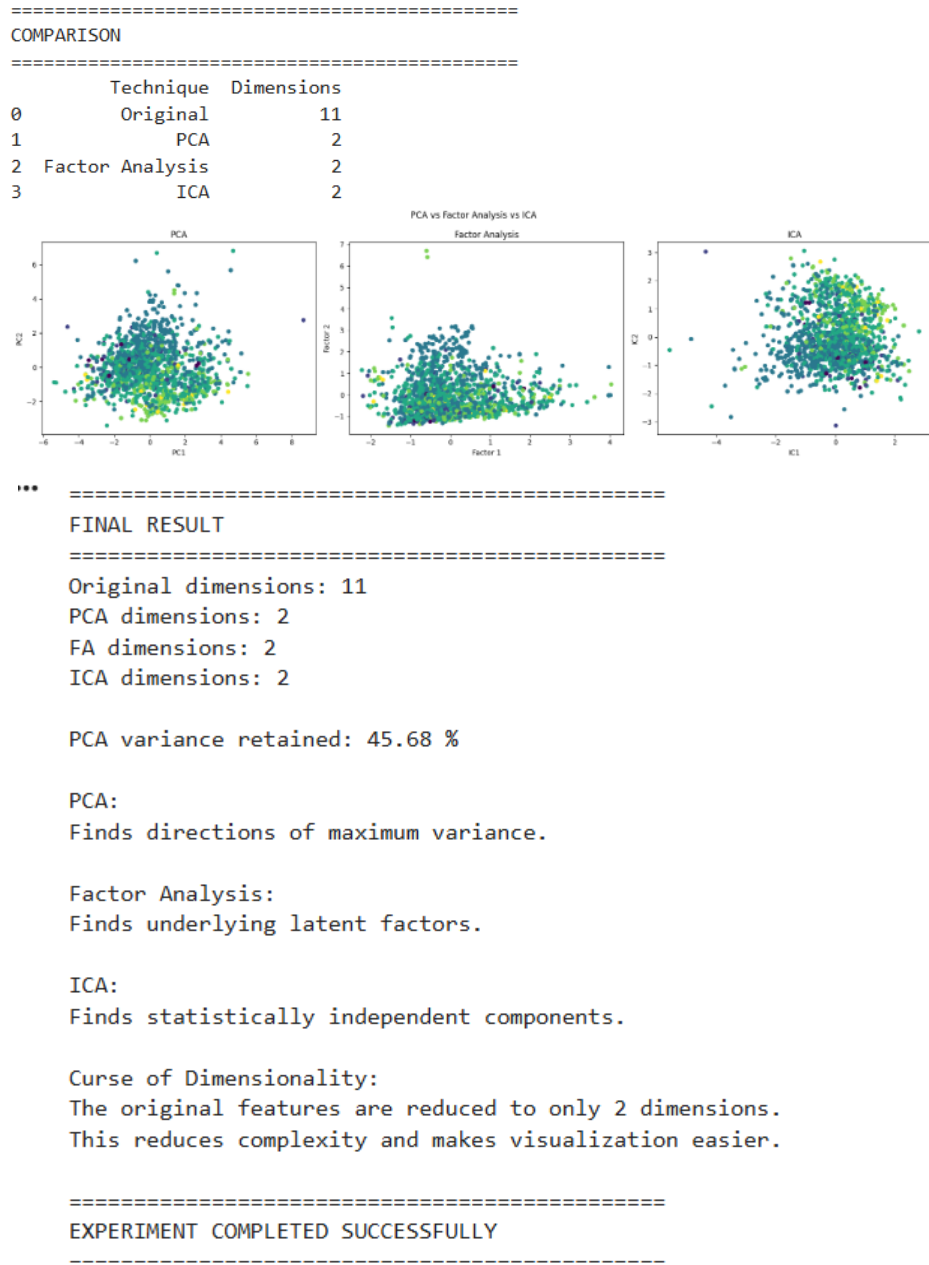
	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	\
IC1	-0.2984	-0.3507	-0.2638	0.1324	-0.0529	
IC2	-0.2199	-0.2136	0.0957	-0.2111	0.1303	

alcohol

IC1 0.2681

IC2 0.0986





Result:

PCA, Factor Analysis, and ICA successfully reduced the dimensionality of the Wine dataset. PCA focuses on maximum variance, FA identifies latent factors, and ICA extracts independent components. Dimensionality reduction decreases the number of features, simplifies visualization and computation, and helps address the Curse of Dimensionality.